Série révision 1
Module : Bases de données avancées
Enseignante responsable : F. BARIKA KTATA

Exercice 1
Soit le schéma relationnel suivant :
Avion (AvNum, AvNom, Capacité, Entrepôt)
Pilote (PiNum, PiNom, PiPrénom, Ville, Salaire, Age, date_embauche, commission)
Vol (VolNum, PiNum, AvNum, VilleDep, VilleArr, HeureDep, HeureArr)

Ecrire un bloc PL/SQL pour chacun des cas suivants :

1. Afficher le nombre de pilote existants dans la base.
2. Afficher le nom de l'avion ayant effectué plus de cinq vols.
3. Afficher le nom et le salaire du premier pilote embauché. Proposer deux solutions différentes, la première en se basant sur les types scalaires et la deuxième sur les types composés.
4. Majorer le salaire de 12% du pilote n°1002 si la valeur de sa commission dépasse la valeur de son salaire. Dans tous les cas, tenir compte d'un message indiquant si son salaire est majoré de 12% ou bien affiché « salaire inférieur à la commission ».
5. Calculer la moyenne des salaires des pilotes dont l'âge est entre 30 et 40 ans.
6. Ecrivez un déclencheur qui, avant un INSERT dans la table Vol, vérifie si un vol retour est présent dans la table (pour que la destination soit desservie dans les 2 sens) et, sinon, affiche un avertissement à la console.

Solution :

```
CREATE OR REPLACE TRIGGER trg_vol_retour
BEFORE INSERT ON Vol
FOR EACH ROW
DECLARE
  v_nb NUMBER;
BEGIN
  SELECT COUNT(*)
  INTO v_nb
  FROM Vol
  WHERE VilleDep = :NEW.VilleArr
    AND VilleArr = :NEW.VilleDep;
  IF v_nb = 0 THEN
    DBMS_OUTPUT.PUT_LINE('Avertissement: aucun vol retour enregistré pour '||
                    :NEW.VilleArr||' -> '||:NEW.VilleDep);
   END IF;
END;
/
```

7. Modifier la table vol de telle sorte que les temps de vol des avions de type A320 (numéro 2 et 3) soient réduits de 8% et ceux des avions de type A330 (12, 13) de 12%.
8. Ecrire une procédure qui permet d'afficher les n premiers pilotes.

Exercice 2
Soit la table suivante :
emp(emp_id, nom, h_d, salaire, #dept_id)

```
CREATE FUNCTION f1(empno NUMBER )
RETURN emp%ROWTYPE
IS
v_req VARCHAR2(200);
v_er emp%ROWTYPE;
BEGIN
v_req := 'SELECT * FROM emp '
|| 'WHERE emp_id = :empno';
EXECUTE IMMEDIATE v_req
USING empno ;
RETURN v_er;
END;
DECLARE v_er emp%ROWTYPE := f1(100);
BEGIN DBMS_OUTPUT.PUT_LINE('Emp: '|| v_er.nom);
END;
```

Ce code comporte-il des erreurs ? Si oui apporter les corrections nécessaires.
Donner son résultat.

Solution :

```
CREATE FUNCTION f1(empno NUMBER)
RETURN emp%ROWTYPE
IS
v_req VARCHAR2(200);
v_er emp%ROWTYPE;
BEGIN
v_req := 'SELECT * FROM emp '
|| 'WHERE emp_id = :empno';
EXECUTE IMMEDIATE v_req INTO v_er
USING empno ;
RETURN v_er;
END;
DECLARE v_er emp%ROWTYPE := f1(100);
BEGIN DBMS_OUTPUT.PUT_LINE('Emp: '|| v_er.nom);
END;
```

Si emp_id=100 existe : affiche Emp: <nom>.
Sinon : exception NO_DATA_FOUND.

Solutionner chacun des cas suivants :

a. Vous devez insérer des données dans la table EMP, mais vous ne savez pas avant la phase d'exécution quelles colonnes seront utilisées. Par exemple, le contenu exact à enregistrer n'est pas connu au moment de l'exécution, précisément c'est au moment de l'appel du script que la décision de l'inclusion ou pas du numéro du département est prise.

```
DECLARE
  v_sql  VARCHAR2(4000);
  v_has_dept BOOLEAN := TRUE; -- supposé paramétré
BEGIN
  IF v_has_dept THEN
    v_sql := 'INSERT INTO emp(emp_id, nom, h_d, salaire, dept_id) VALUES
(:1,:2,SYSDATE,:3,:4)';
    EXECUTE IMMEDIATE v_sql USING 200, 'NOUR', 1800, 10;
  ELSE
    v_sql := 'INSERT INTO emp(emp_id, nom, h_d, salaire) VALUES (:1,:2,SYSDATE,:3)';
    EXECUTE IMMEDIATE v_sql USING 200, 'NOUR', 1800;
  END IF;
END;
/
```

b. A chaque nouvelle insertion d'un employé la date système est affectée à l'attribut h_d.

```
CREATE OR REPLACE TRIGGER trg_emp_hd
BEFORE INSERT ON emp
FOR EACH ROW
BEGIN
  :NEW.h_d := SYSDATE;
END;
/
```

c. L'utilisateur dispose d'une vue contenant la liste des employés affectés aux départements et ayant des grades. Toute modification à travers la vue devra avoir automatiquement un effet sur la base réelle.

```
CREATE OR REPLACE VIEW v_emp_full AS
SELECT e.emp_id, e.nom, e.salaire, d.nomd, g.titre
FROM emp e, departement d, grade g
WHERE e.dept_id = d.dept_id
  AND e.code_grade = g.code_grade;

CREATE OR REPLACE TRIGGER trg_v_emp_full_upd
INSTEAD OF UPDATE ON v_emp_full
FOR EACH ROW
BEGIN
  UPDATE emp
  SET salaire = :NEW.salaire
  WHERE emp_id = :OLD.emp_id;
END;
/
```